

Aufgabenblatt 9

Aufgabe 9.1 iv)

iv) Performance Comparison

Compare the performance of your different evolution step implementations:

- Scalar (single-threaded CPU)
- OpenMP (multi-threaded CPU)
- CUDA (GPU)

Run each implementation on the provided `p67_snark_loop.txt` input for 500,000 iterations. Measure the total execution time using appropriate timing tools. Make sure that you do not print to console and neither have an active delay while benchmarking the different implementations.

Present the results in a table and calculate speedup factors relative to the scalar version.

Then, choose one of the following tasks:

- If available on the machine, use the CUDA profiler to analyze your algorithms performance. Then, copy the plot(s) from the profiler and discuss it in your PDF.
- Compare the algorithm execution runtime for different reasonable number of iterations. Then, plot and discuss the results in your PDF.

```
Soll Parallelisierung verwendet werden?:
```

```
1. Skalar (keine Parallelisierung)
```

```
2. OpenMP
```

```
3. CUDA
```

```
>> 1
```

```
Simulation für 500000 Generationen.
```

```
Gesamtdauer: 57766 Millisekunden.
```

```
Soll Parallelisierung verwendet werden?:
```

```
1. Skalar (keine Parallelisierung)
```

```
2. OpenMP
```

```
3. CUDA
```

```
>> 2
```

```
Simulation für 500000 Generationen.
Gesamtdauer: 16443 Millisekunden.
```

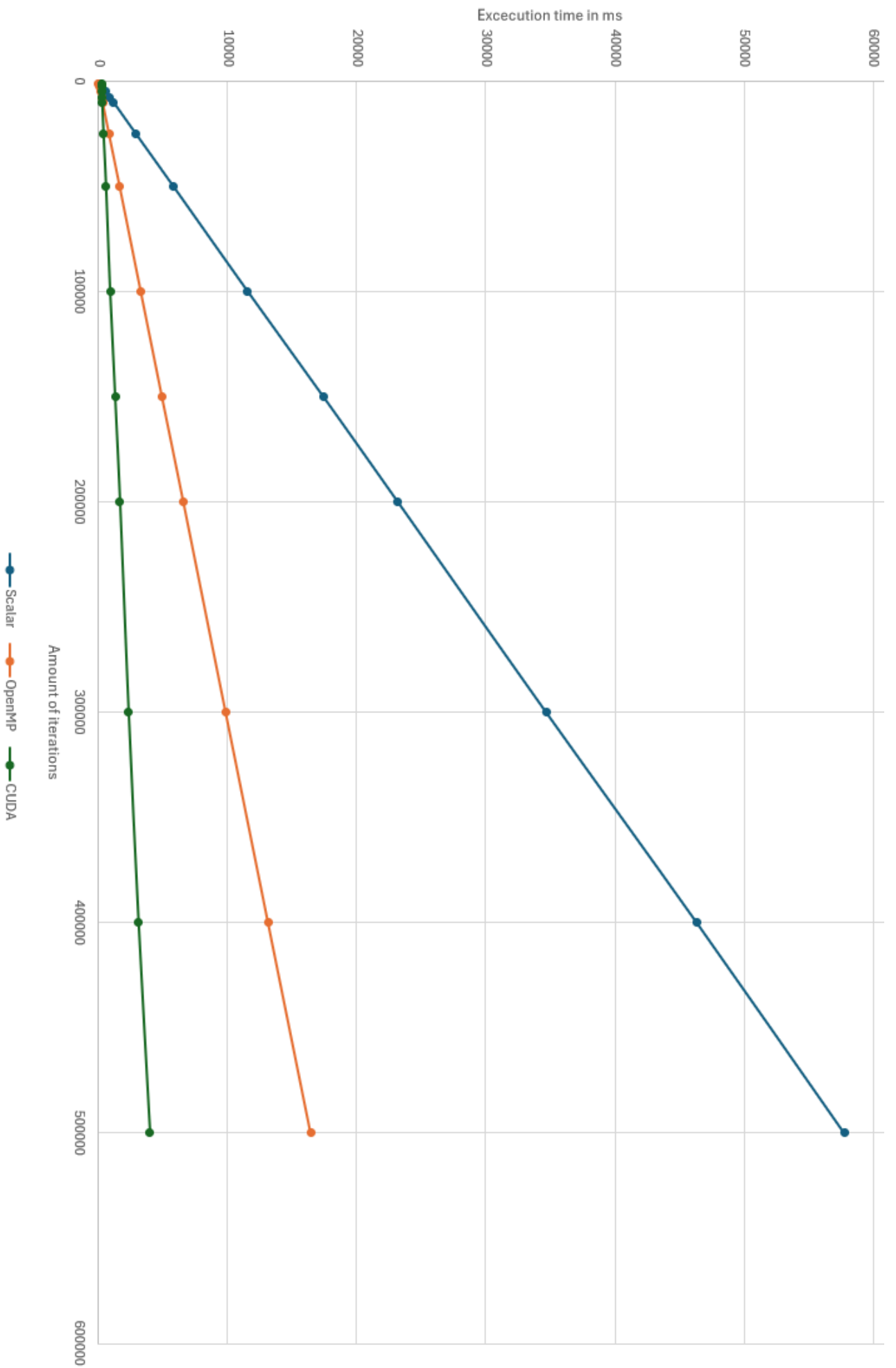
```
Soll Parallelisierung verwendet werden?:
1. Skalar (keine Parallelisierung)
2. OpenMP
3. CUDA
>> 3
```

```
Simulation für 500000 Generationen.
Gesamtdauer: 3972 Millisekunden.
```

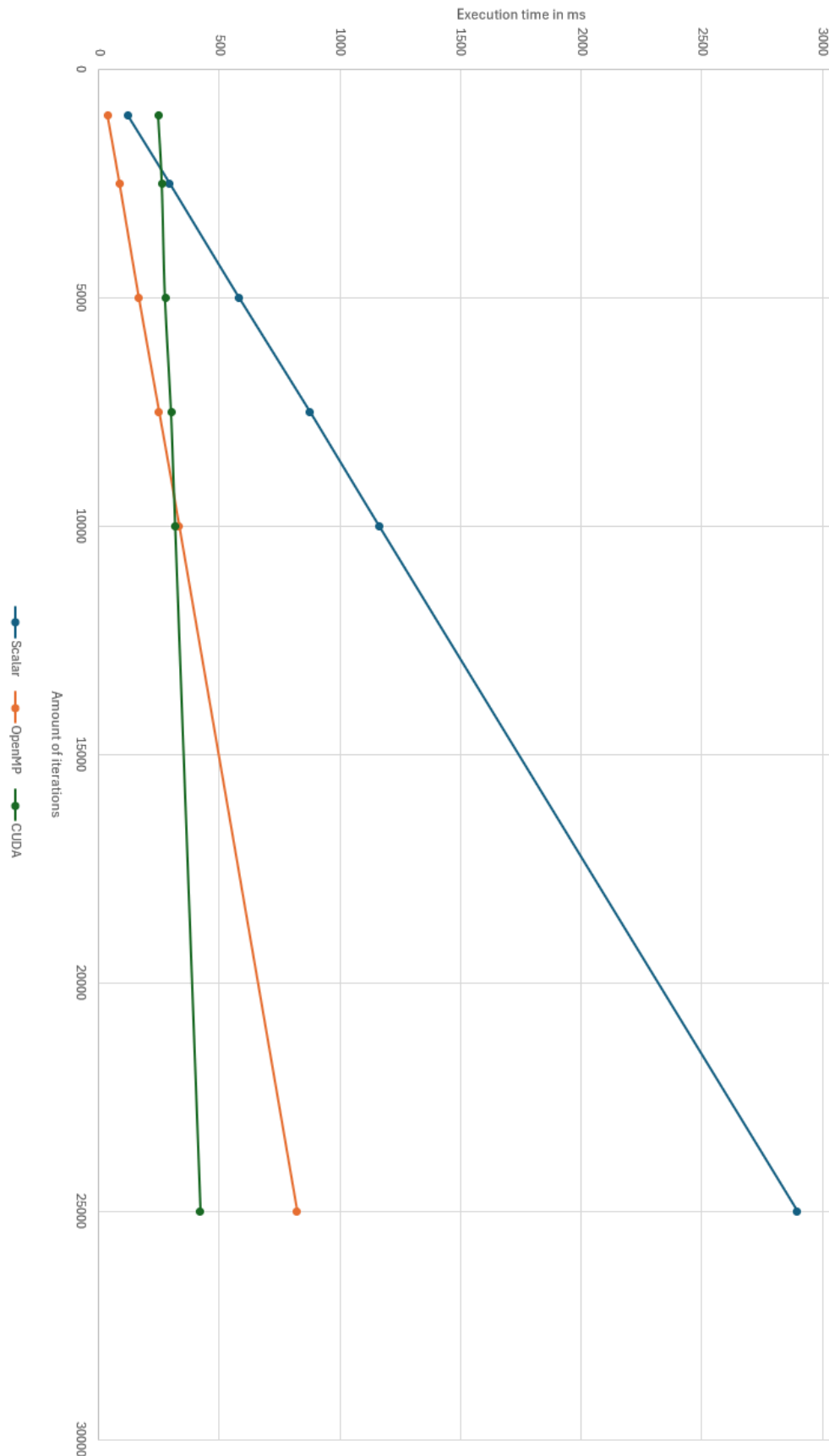
<i>Skalar</i>	<i>OpenMP</i>	<i>CUDA</i>
- Laufzeit: 57766 ms	- Laufzeit: 16443 ms	- Laufzeit: 3972 ms
- (1x Speedup)	- 3,5x Speedup	- 14,5x Speedup

Iterationen	Laufzeit in ms			Speedup von OpenMP	Speedup von CUDA
	Scalar	OpenMP	CUDA		
1000	120	37	248	3.243243243	0.483870968
2500	295	87	263	3.390804598	1.121673004
5000	582	167	276	3.48502994	2.108695652
7500	876	251	301	3.490039841	2.910299003
10,000	1162	334	318	3.479041916	3.65408805
25,000	2894	821	422	3.524969549	6.857819905
50,000	5799	1649	610	3.516676774	9.506557377
100,000	11566	3285	924	3.520852359	12.51731602
150,000	17421	4926	1305	3.536540804	13.34942529
200,000	23190	6586	1679	3.521105375	13.81179273
300,000	34679	9862	2340	3.516426688	14.82008547
400,000	46348	13148	3094	3.525098874	14.97996122
500,000	57766	16443	3972	3.513105881	14.54330312

Plot auf der nächsten Seite:



Wenn man hineinzoomt, erkennt man den Overhead von CUDA:



Erklärungen für die Ergebnisse

Beobachtung 1: CUDA hat bei 500000 Generationen einen 14,5x Speedup. Müsste das nicht viel größer sein? 10000x Speedup?

- Pro Generation macht jeder Thread sehr wenig Arbeit (ein paar einfache Operationen), daher wird die Rechenzeit von Overhead dominiert.
- Man braucht pro Zelle mehrere Speicherzugriffe (z. B. auf 8 Nachbarn). Das limitiert die Geschwindigkeit mehr als Rechenleistung.
- Kernel-Launch und Kopierkosten (von RAM zu VRAM) haben Einfluss auf den Speedup insgesamt.

Beobachtung 2: CUDA ist für kleine Iterationszahlen langsamer als OpenMP und sogar Skalar. Warum?

- Bei nur 1000 Generationen macht die Kernel-Launch-Zeit und die Kopierzeit einen relativ großen Anteil der Laufzeit aus => Overhead lohnt sich nicht für die kurze Rechenzeit.
- Für 100x100 Gitter oder kleiner ist der CPU-Cache gut. Die CPU kann das schnell ausführen ohne nennenswerte Latenz.